UniGe | MaLGa

# MLCC 2026

# Dimensionality Reduction (and maybe... Clustering!)

Cesare Molinari

MaLGa, UniGe-Dima

# Outline

**PCA & Reconstruction**

**Kernel PCA**

**Auto-encoders**

# Dimensionality Reduction

In many practical applications we want to reduce the dimensionality of the data:

- Data Visualization
- Data Exploration (investigate the “effective” data dimensionality)

# What do we want?

**Dimensionality Reduction:** find a map

$$M : \mathbb{R}^d \to \mathbb{R}^k \qquad \text{with} \quad k \ll d,$$

according to some *suitable criterion*...

In the following, our guiding principle: **data reconstruction**!

# Data Reconstruction

Find a **dimensionality-reduction map**

$$M : \mathbb{R}^d \to \mathbb{R}^k \qquad \text{with} \quad k \ll d,$$

and a **reconstruction map** $R : \mathbb{R}^k \to \mathbb{R}^d$ s.t.

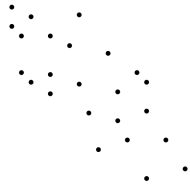
- if $x_i \in \mathbb{R}^d$
- if $v_i = M(x_i)$
- and $\tilde{x}_i = R(v_i)$

then

$$\|\tilde{x}_i - x_i\| \quad \text{is small}$$

# Principal Component Analysis

**PCA**: most popular dimensionality reduction procedure



What do we have? **Unsupervised** samples, $S = (x_1, \ldots, x_n)$

What do we want? A **data driven** procedure to get a linear map $M$ **preserving the data structure**

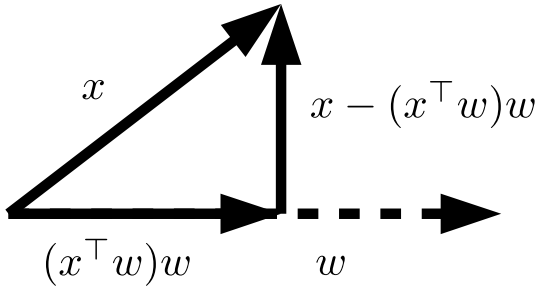
Today: a **geometric** derivation of PCA

# Dimensionality Reduction by Reconstruction

If

$$w \in \mathbb{R}^d \qquad \text{with} \quad \|w\| = 1,$$

then $(w^\top x)w$ is the **orthogonal projection** of $x$ on $w$



# Dimensionality Reduction by Reconstruction

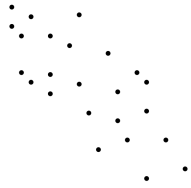
First consider $k = 1$ ($Mx = w^\top x$), with **reconstruction map** $R(v) = vw$ and **reconstruction error**

$$\|x - (w^\top x)w\|^2$$

(that is how much we lose by projecting $x$ along the direction $w$)

**Problem:**
Find the direction $p$ allowing the best reconstruction of the training set!

# Dimensionality Reduction by Reconstruction



- $\mathbb{S}^{d-1} = \{w \in \mathbb{R}^d : \ \|w\| = 1\}$ is the sphere in dimension $d$

Consider the **empirical reconstruction** minimization problem

$$\min_{w \in \mathbb{S}^{d-1}} \ \frac{1}{n} \sum_{i=1}^{n} \|x_i - (w^\top x_i) w\|^2$$

The solution $p$ is called **first principal component** of the data

# An Equivalent Formulation

**Pythagoras's theorem:**

$$\|x_i - (w^\top x_i)w\|^2 = \|x_i\|^2 - (w^\top x_i)^2$$

Then,

$$\min_{w \in \mathbb{S}^{d-1}} \frac{1}{n} \sum_{i=1}^{n} \|x_i - (w^\top x_i)w\|^2$$

is equivalent to

$$\max_{w \in \mathbb{S}^{d-1}} \frac{1}{n} \sum_{i=1}^{n} (w^\top x_i)^2$$

# Reconstruction and Variance

If the data is centered ($\frac{1}{n}\sum_{i=1}^{n} x_i = 0$),

$$(w^\top x)^2$$

is the **variance** of $x$ in the direction $w$

**First PC:** direction along which the data have **maximum variance**

$$\max_{w \in \mathbb{S}^{d-1}} \quad \frac{1}{n}\sum_{i=1}^{n}(w^\top x_i)^2$$

# Centering

If the data are not centered, consider

$$\max_{w \in \mathbb{S}^{d-1}} \frac{1}{n} \sum_{i=1}^{n} (w^\top (x_i - \bar{x}))^2$$

equivalent to

$$\max_{w \in \mathbb{S}^{d-1}} \frac{1}{n} \sum_{i=1}^{n} (w^\top x_i^c)^2$$

with $x^c = x - \bar{x}$

# Centering and Reconstruction

Or, consider the effect of centering to reconstruction, we get

$$\min_{w \in \mathbb{S}^{d-1},\ b \in \mathbb{R}^d} \quad \frac{1}{n} \sum_{i=1}^{n} \|x_i - ((w^\top (x_i - b))w + b)\|^2$$

where

$$((w^\top (x_i - b))w + b$$

is an **affine** (rather than linear) map

# PCA as an Eigenproblem

Manipulating,

$$\frac{1}{n}\sum_{i=1}^{n}(w^\top x_i)^2 = \frac{1}{n}\sum_{i=1}^{n} w^\top x_i w^\top x_i = \frac{1}{n}\sum_{i=1}^{n} w^\top x_i x_i^\top w = w^\top \left(\frac{1}{n}\sum_{i=1}^{n} x_i x_i^\top\right) w$$

Then, we can consider the problem

$$\max_{w\in\mathbb{S}^{d-1}} \quad w^\top C w, \qquad \text{where} \quad C = \frac{1}{n}\sum_{i=1}^{n} x_i x_i^\top$$

# PCA as an Eigenproblem (cont.)

- The **covariance matrix** $C = \frac{1}{n} \sum_{i=1}^{n} x_i x_i^\top$ is symmetric and semi-definite positive
- PCA can be written as a *Rayleigh* quotient:

$$\max_{w \in \mathbb{R}^d} \frac{w^\top C w}{w^\top w}$$

- If $Cu = \lambda u$, then $\frac{u^\top C u}{u^\top u} = \lambda$.

*In fact, the Rayleigh quotient achieves its maximum at any eigenvector corresponding to the maximum eigenvalue of $C$*

# PCA as an Eigenproblem

Computing the first principal component of the data reduces to computing the biggest eigenvalue of the covariance and the corresponding eigenvector...

$$Cu = \lambda u, \qquad \text{where} \quad C = \frac{1}{n} \sum_{i=1}^{n} x_i x_i^\top$$

# Beyond the First Principal Component

How to consider more than one principle component ($k > 1$)?

$$M : \mathbb{R}^d \to \mathbb{R}^k, \quad k \ll d$$

**Idea:** simply iterate the previous reasoning...

# Residual Reconstruction

Consider the one dimensional projection that best reconstruct the residuals

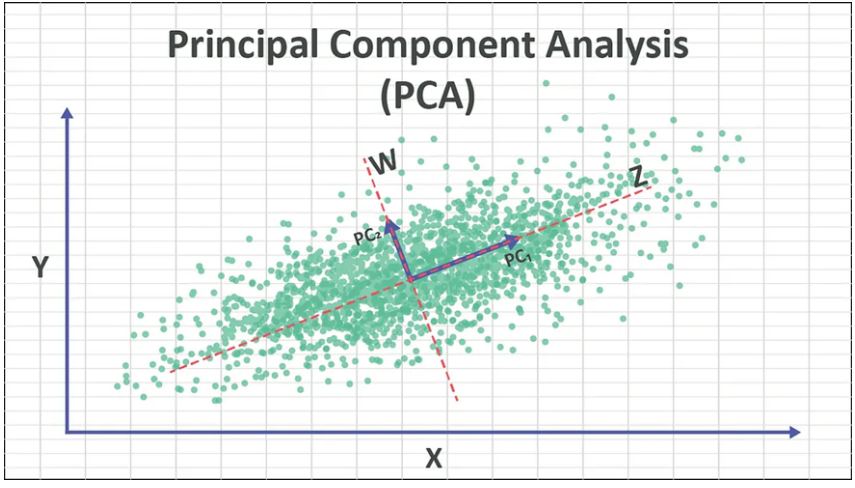
$$r_i = x_i - (p^\top x_i)p$$

Associated minimization problem:

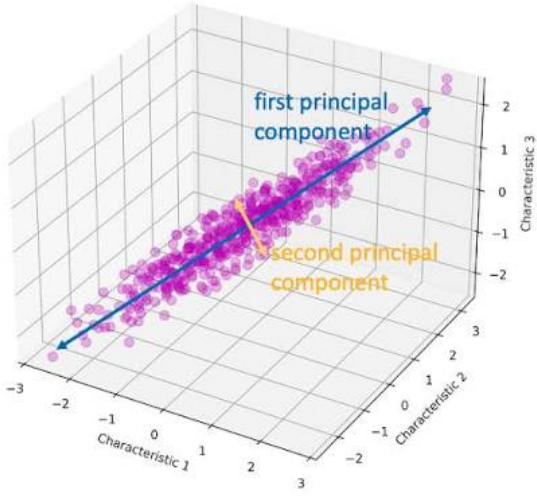
$$\min_{w \in \mathbb{S}^{d-1},\ w \perp p} \frac{1}{n} \sum_{i=1}^{n} \|r_i - (w^\top r_i)w\|^2$$

(note the constraint $w \perp p$)

# PCA



# PCA


first principal component
second principal component
Characteristic 1
Characteristic 2
Characteristic 3

# Residual Reconstruction

Since $w \perp p$,

$$\|r_i - (w^\top r_i)w\|^2 = \|r_i\|^2 - (w^\top r_i)^2 = \|r_i\|^2 - (w^\top x_i)^2$$

Then,

$$\max_{w \in \mathbb{S}^{d-1},\ w \perp p} \quad \frac{1}{n}\sum_{i=1}^{n}(w^\top x_i)^2 = w^\top C w$$

# PCA as an Eigenproblem

Again, maximize Rayleigh quotient of $C$ (with extra constraint $w \perp p$)

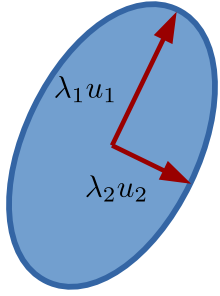
$$\max_{w \in \mathbb{S}^{d-1},\ w \perp p} \quad \frac{1}{n} \sum_{i=1}^{n} (w^\top x_i)^2 = w^\top C w$$

It can be proven that the solution of the above problem is given by the second eigenvector of $C$

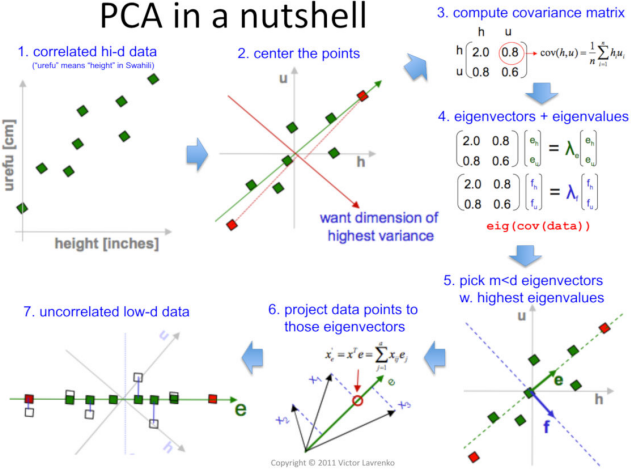
# PCA as an Eigenproblem

The reasoning generalizes to more than two components: computation of $k$ PCs $(p_1, ..., p_k)$ for data $(x_1, ..., x_n)$ corresponds to finding first $k$ eigenvectors $(u_1, ..., u_k)$ of $C$

$$Cu = \lambda u, \qquad \text{where} \quad C = \frac{1}{n}\sum_{i=1}^{n} x_i x_i^\top$$



# PCA


PCA in a nutshell
1. correlated hi-d data
("urefu" means "height" in Swahili)
urefu [cm]
height [inches]
2. center the points
u
h
want dimension of highest variance
3. compute covariance matrix
h u
h 2.0 0.8
u 0.8 0.6
$\text{cov}(h,u) = \frac{1}{n}\sum_{i=1}^{n} h_i u_i$
4. eigenvectors + eigenvalues
$\begin{pmatrix} 2.0 & 0.8 \\ 0.8 & 0.6 \end{pmatrix} \begin{pmatrix} e_h \\ e_u \end{pmatrix} = \lambda_e \begin{pmatrix} e_h \\ e_u \end{pmatrix}$
$\begin{pmatrix} 2.0 & 0.8 \\ 0.8 & 0.6 \end{pmatrix} \begin{pmatrix} f_h \\ f_u \end{pmatrix} = \lambda_f \begin{pmatrix} f_h \\ f_u \end{pmatrix}$
eig(cov(data))
5. pick m<d eigenvectors
w. highest eigenvalues
u
h
e
f
6. project data points to
those eigenvectors
$x'_e = x^T e = \sum_{j=1}^{d} x_j e_j$
Copyright © 2011 Victor Lavrenko
7. uncorrelated low-d data
e

# PCA as an Eigenproblem

The reasoning generalizes to more than two components: computation of $k$ PCs $(p_1, ..., p_k)$ for data $(x_1, ..., x_n)$ corresponds to finding first $k$ eigenvectors $(u_1, ..., u_k)$ of $C$

$$M: \ x \in \mathbb{R}^d \longmapsto Mx = \begin{bmatrix} p_1^\top x \\ ... \\ p_k^\top x \end{bmatrix} \in \mathbb{R}^k$$

and

$$R: \ v \in \mathbb{R}^k \longmapsto Rv = \sum_{i=1}^{k} v_j p_j \in \mathbb{R}^d$$

So

$$M = \begin{bmatrix} p_1^\top \\ ... \\ p_k^\top \end{bmatrix} \qquad \text{and} \qquad R = M^\top$$

# Remarks

- Computational complexity: roughly $O(kd^2)$ - forming $C$ is $O(nd^2)$
- The dimensionality reduction induced by PCA is a linear projection. Can we generalize PCA to non linear dimensionality reduction?

# Singular Value Decomposition (SVD)

Consider the SVD of the (normalized) data matrix $X$

$$\frac{1}{\sqrt{n}}X = U\Sigma V^\top$$

where

- $U$ is a $n$ by $k$ orthogonal matrix
- $V$ is a $d$ by $k$ orthogonal matrix
- $\Sigma$ is a diagonal matrix s.t. $\Sigma_{j,j} = \sigma_j^2$, $j = 1, \dots, k$ and $k \leq \min\{n, d\}$.

The columns of $U$ and the columns of $V$ are the left and right singular vectors and the diagonal entries of $\Sigma$ the singular values

# Singular Value Decomposition

Observe that

$$C = \frac{1}{n}\sum_{i=1}^{n} x_i x_i^\top = \frac{1}{n} X^\top X = (\frac{1}{\sqrt{n}} X^\top)(\frac{1}{\sqrt{n}} X) = V\Sigma^2 V^\top$$

$$K := \frac{1}{n} X X^\top = (\frac{1}{\sqrt{n}} X)(\frac{1}{\sqrt{n}} X^\top) = U\Sigma^2 U^\top$$

The SVD can be equivalently described by the equations

$$Cp_j = \sigma_j^2 p_j$$
$$Ku_j = \sigma_j^2 u_j$$
$$Xp_j = \sigma_j u_j$$
$$X^\top u_j = \sigma_j p_j$$

# PCA and Singular Value Decomposition

If $n \ll d$, we can consider the following procedure:

- form the matrix $K$, which is $O(dn^2)$
- find the first $k$ eigenvectors $(u_1, ..., u_k)$ of $K$, which is $O(kn^2)$
- compute the principal components using

$$p_j = \frac{1}{\sigma_j} X^\top u_j, \quad j = 1, \ldots, k$$

# Outline

PCA & Reconstruction

**Kernel PCA**

Auto-encoders

# Beyond Linear Dimensionality Reduction?

By considering PCA we are implicitly assuming the data to lie on a linear subspace....

...it is easy to think of situations where this assumption might violated.

Can we use kernels to obtain non linear generalization of PCA?

# From SVD to KPCA

The projection of a point $x$ on a principal component $p_j$ is

$$(M(x))^j = x^\top p_j = \frac{1}{\sigma_j} x^\top X^\top u_j = \frac{1}{\sigma_j} \sum_{i=1}^{n} x^\top x_i u_j^i,$$

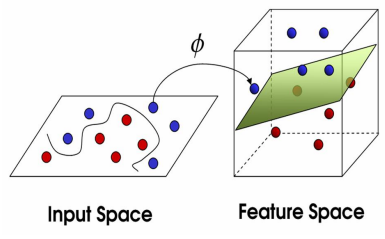
Recall

$$Cp_j = \sigma_j^2 p_j, \quad Ku_j = \sigma_j^2 u_j,$$
$$Xp_j = \sigma_j u_j, \quad X^\top u_j = \sigma_j p_j$$

# PCA and Feature Maps

$$(M(x))^j = \frac{1}{\sigma_j} \sum_{i=1}^{n} x^\top x_i u_j^i,$$

What if consider a non linear feature-map $\Phi : X \to F$, before performing PCA?




$$(M(x))^j = \Phi(x)^\top p_j = \frac{1}{\sigma_j} \sum_{i=1}^{n} \Phi(x)^\top \Phi(x_i) u_j^i$$

# Kernel PCA

Repeat:

$$(M(x))^j = \Phi(x)^\top p_j = \frac{1}{\sigma_j} \sum_{i=1}^{n} \Phi(x)^\top \Phi(x_i) u_j^i,$$

If the feature map is defined by a positive definite kernel $K\colon\ X \times X \to \mathbb{R}$, then

$$(M(x))^j = \frac{1}{\sigma_j} \sum_{i=1}^{n} K(x, x_i) u_j^i$$

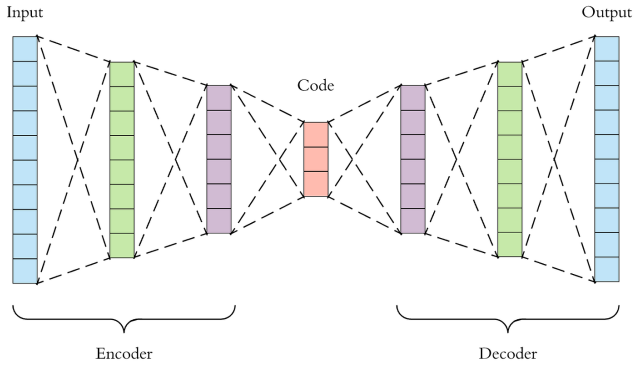
# Outline

PCA & Reconstruction

Kernel PCA

Auto-encoders

# Auto-encoders




- A NN with input and output layers both in $\mathbb{R}^d$
- Middle bottleneck in $\mathbb{R}^k$ (coding)
- Trained to **predict the input** (mimic the identity)

**Example:** auto-encoder with one hidden layer

$$\Psi : \mathbb{R}^d \to \mathbb{R}^k, \qquad \Psi(x) = \sigma(Wx)$$

and

$$\Phi : \mathbb{R}^k \to \mathbb{R}^d, \qquad \Phi(\beta) = \sigma\left(W'\beta\right)$$

Train: for $\theta$ the set of parameters,

$$\min_{\theta} \sum_{i=1}^{n} \|\Phi_\theta(\Psi_\theta(x_i)) - x_i\|^2$$

# Maintaining the relative distances

**Johnson–Lindenstrauss Lemma**

Given $0 < \varepsilon < 1$, a set $X = (x_1, ..., x_n)$ of $n$ points in $\mathbb{R}^d$, and an integer $k > \frac{8 \ln n}{\varepsilon^2}$, there is a linear map $f : \mathbb{R}^d \to \mathbb{R}^k$ such that

$$(1 - \varepsilon)\|u - v\|^2 \leq \|f(u) - f(v)\|^2 \leq (1 + \varepsilon)\|u - v\|^2$$

for all $u, v \in X$

Meaning: a set of points in a high-dimensional space $\mathbb{R}^d$ can be embedded **linearly** into a space of much lower dimension $\mathbb{R}^k$ so that **distances between the points are nearly preserved**

# Wrapping Up

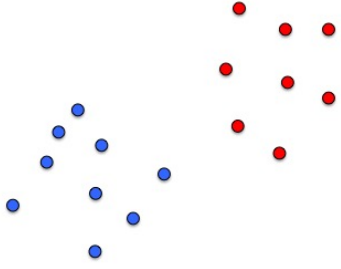
In this class we introduced PCA as a basic tool for dimensionality reduction. We discussed computational aspect and extensions to non linear dimensionality reduction (KPCA and Auto-Encoders)

## About next class

We will consider an *unsupervised setting*, and in particular the problem of **clustering** unlabeled data into “coherent” groups

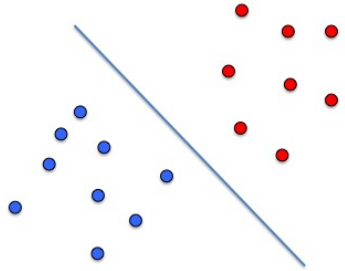
# Supervised learning

- "Learning with a teacher"
- Data set $S = \{(x_1, y_1), \ldots, (x_n, y_n)\}$ with $x_i \in \mathbb{R}^d$ and $y_i \in \mathbb{R}$
- $X = (x_1, \ldots, x_n)^\top \in \mathbb{R}^{n \times d}$ and $y = (y_1, \ldots, y_n)^\top$



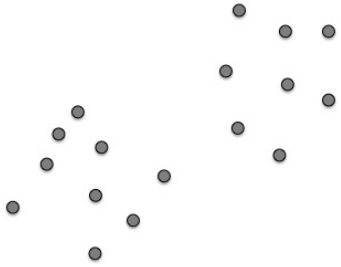
# Supervised learning

- "Learning with a teacher"
- Data set $S = \{(x_1, y_1), \ldots, (x_n, y_n)\}$ with $x_i \in \mathbb{R}^d$ and $y_i \in \mathbb{R}$
- $X = (x_1, \ldots, x_n)^\top \in \mathbb{R}^{n \times d}$ and $y = (y_1, \ldots, y_n)^\top$



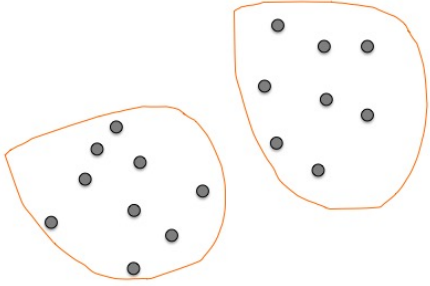
# Unsupervised learning

- "Learning without a teacher"
- Data set $S = \{x_1, \ldots, x_n\}$ with $x_i \in \mathbb{R}^d$



# Unsupervised learning

- "Learning without a teacher"
- Data set $S = \{x_1, \ldots, x_n\}$ with $x_i \in \mathbb{R}^d$



# Unsupervised learning problems

- Dimensionality reduction
- **Clustering**
- Density estimation
- Learning association rules
- Learning adaptive data representations
- ...

# Supervised vs unsupervised methods

- **Supervised learning:** measure of success (loss function and cross-validation)
- **Unsupervised learning:** not!
  - proliferation of many heuristic algorithms difficult to evaluate — lack of theoretical grounds

# Clustering

- *Clustering* is a widely used technique for *data analysis* (applications in statistics, computer science, biology, social sciences...)
- **Goal:** grouping/segmenting a collection of objects into subsets or clusters

# Combinatorial clustering

- We assume some knowledge on the number of clusters $K \leq n$.
  *Goal:* associate a cluster label $k \in \{1, \ldots, K\}$ with each datum, by defining an encoder $\mathcal{C}$ s.t.

$$k = \mathcal{C}(x_i)$$

- We look for an encoder $\mathcal{C}^*$ that achieves the goal of clustering data, according to some specific requirement of the algorithm and based on data pairs dissimilarities

## Combinatorial clustering

- *Criterion:* assign to the same cluster similar/close data
- Energy function (within class):

$$W(\mathcal{C}) \;=\; \sum_{k=1}^{K} \sum_{\mathcal{C}(i)=k,\ \mathcal{C}(i')=k} d(x_i, x_{i'})$$

- $\mathcal{C}^* \in \arg\min\ W(\mathcal{C})$
- BUT... unfeasible in practice!

$$S(n, K) = \frac{1}{K!} \sum_{k=1}^{K} (-1)^{K-k} \binom{K}{k} k^n$$

and notice that $S(10, 4) \sim 34k$ while $S(19, 4) \sim 10^{10}$

# K-means algorithm

- Initialize at random cluster centroids $m_k$ for $k = 1, \ldots, K$
- repeat until convergence
  1. assign data to centroids $\mathcal{C}(x_i) = \arg\min_{1 \leq k \leq K} ||x_i - m_k||^2$
  2. update centroids

# K-means functional

K-means corresponds to minimizing the following function

$$J(\mathcal{C}, m) = \sum_{k=1}^{K} \sum_{\mathcal{C}(i)=k} ||x_i - m_k||^2$$

The algorithm is an alternating optimization procedure, with convergence guarantees (no rates)

BUT... the function $J$ is not convex, thus K-means is not guaranteed to find a global minimum

Computational cost

1. data assignment $O(Kn)$
2. cluster centers updates $O(n)$

# K-means

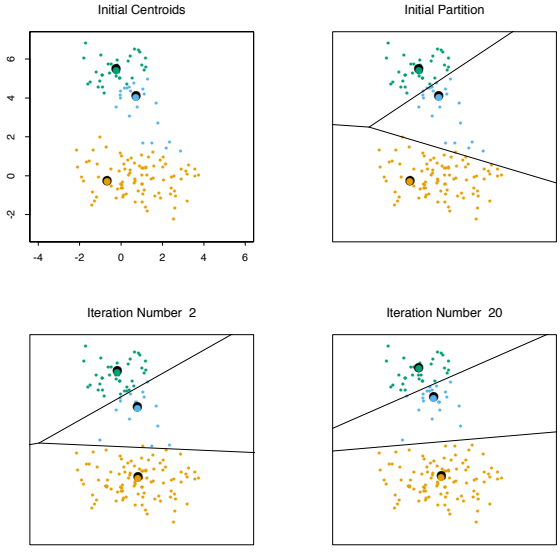



Figure from Hastie, Tibshirani, Friedman

# Wrapping up

In this class we introduced the concept of data clustering and sketched some of the best known algorithms